Aim:

Write a program to implement the 0/1 Knapsack Problem. Given N items, each having a value and a weight, and a knapsack with capacity W, determine the maximum total value that can be obtained by selecting a subset of the items such that the total weight does not exceed W. An item can either be included completely or excluded completely from the knapsack. Fractional selection of items is not allowed.

Input Format:

- The first line contains an integer N, representing the number of items.
- The second line contains N space-separated integers representing the values of the items.
- The third line contains N space-separated integers representing the weights of the items.
- The fourth line contains an integer W, representing the capacity of the knapsack.

Output Format:

- Print a single integer representing the maximum value that can be obtained without exceeding the knapsack capacity.

Constraints:

- $1 \leq N \leq 1000$
- $1 \leq W \leq 1000$
- $1 \leq value[i], weight[i] \leq 1000$

Sample Input:

```
3
1 2 3
4 5 1
4
```

Sample Output:

```
3
```

Explanation: The knapsack capacity is 4. Among the given items, selecting the third item (value = 3, weight = 1) yields the maximum value without exceeding the capacity. Therefore, the output is 3.

Note:

- Each item can be selected at most once.
- Fractional selection of items is not allowed.
- The total weight of the selected items must not exceed the knapsack capacity W.

Source Code:

maxValueInKnapsack.c

```c
#include <stdio.h>
#include <stdlib.h>

// Helper function to return the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *values = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &values[i]) != 1) {
            free(values);
            return 0;
        }
    }

    int *weights = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &weights[i]) != 1) {
            free(values);
            free(weights);
            return 0;
        }
    }

    int W;
    if (scanf("%d", &W) != 1) {
        free(values);
        free(weights);
        return 0;
    }

    // Allocate memory for the DP state array and initialize all elements to 0
    int *dp = (int *)calloc(W + 1, sizeof(int));

    // Outer loop processes each item sequentially
    for (int i = 0; i < n; i++) {
        // Inner loop travels backwards from W down to the current item's weight
        // Traveling backwards ensures we only count each item at most once
        for (int j = W; j >= weights[i]; j--) {
            dp[j] = max(dp[j], dp[j - weights[i]] + values[i]);
        }
    }

    // Output the maximum possible value matching strict platform whitespace criteria
    printf("%d\n", dp[W]);

    // Deallocate memory blocks
    free(values);
    free(weights);
    free(dp);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
3
1 2 3
4 5 1
4
3

Test Case - 2
User Output
3
1 2 3
4 5 6
3
0